1. Separate Search Engine Logic from UI Component

Currently, functions like `matchesWordBoundary`, `rankResources`, and `matchDirectoryQuery` live inside the component file or compute on state triggers. Moving this logic to a standalone utility module improves testability and readability.

- **Create `src/utils/searchEngine.ts`:** Extract all query-matching logic, ranking algorithms, and search categories into a dedicated file.
- **Benefits:**
  - Keeps `CivicChatbot.tsx` focused purely on rendering UI and managing chat interactions.
  - Allows you to write unit tests for `rankResources` and `matchesWordBoundary` using tools like Vitest or Jest without mounting React components.

2. Standardize Category Intent Rules

Instead of using large chains of `if` statements with hardcoded `matchesWordBoundary` calls, map topics to standard configuration objects.

TypeScript

```
None
// src/utils/searchEngine.ts
interface TopicRule {
  id: string;
  category: string;
  keywords: string[];
  responseTitle: string;
  examplePrompts: string[];
}

const TOPIC_RULES: TopicRule[] = [
  {
    id: 'food',
    category: 'food',
    keywords: ['food', 'pantry', 'pantries', 'groceries',
'grocery', 'hunger', 'meal', 'meals', 'snap', 'ebt'],
    responseTitle: "Need groceries this week? Multiple pantries in
Cumming distribute fresh food and baby supplies.",
    examplePrompts: [
```

```
      'Which food pantries are open on weekday evenings?',
      'Can I get food boxes delivered to my door for my kids?',
      'Where can I find free baby formula and diapers in Forsyth?'
    ]
  },
  // ... remaining category configs
];
```

3. Implement Auto-Focus and Accessibility (a11y)

- **Auto-focusing on Open:** Add a `ref` to the text input so it automatically receives focus when the user opens the chatbot widget.
- TypeScript

```
None
const inputRef = useRef<HTMLInputElement>(null);

useEffect(() => {
  if (isOpen) {
    scrollToBottom();
    inputRef.current?.focus();
  }
}, [isOpen]);
```

-
- **Accessible Dialog Attributes:**
  - Add `aria-expanded={expandedResources.has(res.id)}` to the resource accordion buttons.
  - Ensure the chat message area uses `aria-live="polite"` so screen readers read out new incoming response messages automatically.

4. Optimize Response Scrolling & UX Features

- **Expanded View Scroll Adjustment:** When a user toggles **Learn More** on a resource card inside a message bubble, the message list height expands, which can push content off-screen. Adding `scrollToBottom()` or smooth scroll adjustments on `expandedResources` state changes keeps the view focused.

- **Header Reset Option:** Add a "Reset / Clear Chat" icon button next to the close button in the header so users can clear long conversation threads back to the initial welcome state.

5. Architectural Alignment with Your Project Structure

Looking at your repository structure (`src/components`, `src/hooks`, `src/data`):

- **Create a Custom Hook (`src/hooks/useCivicChat.ts`):** Move state handling (`messages`, `isTyping`, `lastTopic`, `lastMatchedResources`, `nextResourceIndex`) out of the presentation layer into a clean custom hook:
- TypeScript

```
None
const { messages, isTyping, sendMessage, clearChat } =
useCivicChat();
```

-
- **Reuse Data Types:** Ensure your message-matching fallback types align with `ResourceItem` from `src/types/index.ts` to maintain consistency across `ResourceCard.tsx`, `ResourceDetails.tsx`, and `CivicChatbot.tsx`.

1. Install Supabase Client

In your project terminal, install the JS client:

Bash

```
None
npm install @supabase/supabase-javascript
```

2. Initialize Supabase (`src/lib/supabase.ts`)

Create a client helper with your project credentials (store keys in `.env`):

TypeScript

```
None
import { createClient } from '@supabase/supabase-javascript';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
```

```
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

3. Update Feedback Hook / Component (`src/hooks/useFeedback.ts`)

Listen to real-time `INSERT` changes on your `feedback` table. When anyone submits feedback anywhere, every online browser automatically updates its state.

TypeScript

```
None
import { useEffect, useState } from 'react';
import { supabase } from '../lib/supabase';

export interface FeedbackItem {
  id: string;
  name: string;
  comment: string;
  created_at: string;
}

export const useFeedback = () => {
  const [feedbacks, setFeedbacks] = useState<FeedbackItem[]>([]);

  // 1. Fetch existing feedbacks on initial load
  useEffect(() => {
    const fetchFeedback = async () => {
      const { data, error } = await supabase
        .from('feedback')
        .select('*')
        .order('created_at', { ascending: false });

      if (data) setFeedbacks(data);
    };

    fetchFeedback();
```

```
    // 2. Subscribe to real-time updates across all browsers
    const channel = supabase
      .channel('public:feedback')
      .on(
        'postgres_changes',
        { event: 'INSERT', schema: 'public', table: 'feedback' },
        (payload) => {
          const newFeedback = payload.new as FeedbackItem;
          setFeedbacks((prev) => [newFeedback, ...prev]);
        }
      )
      .subscribe();

    // Cleanup subscription when component unmounts
    return () => {
      supabase.removeChannel(channel);
    };
  }, []);

  // Function to submit feedback
  const addFeedback = async (name: string, comment: string) => {
    await supabase.from('feedback').insert([{ name, comment }]);
  };

  return { feedbacks, addFeedback };
};
```

Option 2: Socket.io (If you have a Node.js / Express Backend)

If your GitHub repository includes a custom Express/Node server, you can set up a Socket.io server to broadcast events.

1. Server Setup (`server.js`)

JavaScript

```
None
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  // Listen for feedback submissions
  socket.on('submit_feedback', (newFeedback) => {
    // Broadcast the new feedback to ALL connected clients
    io.emit('feedback_received', newFeedback);
  });

  socket.on('disconnect', () => {
    console.log('User disconnected');
  });
});

server.listen(4000, () => console.log('Socket server running on
port 4000'));
```

2. React Client Setup

Bash

```
None
npm install socket.io-client
```

TypeScript

```
None
import { useEffect, useState } from 'react';
import { io } from 'socket.io-client';
```

```
const socket = io('http://localhost:4000'); // Your backend URL

export const FeedbackSection = () => {
  const [feedbacks, setFeedbacks] = useState<any[]>([]);

  useEffect(() => {
    // Listen for broadcasted feedback from server
    socket.on('feedback_received', (newFeedback) => {
      setFeedbacks((prev) => [newFeedback, ...prev]);
    });

    return () => {
      socket.off('feedback_received');
    };
  }, []);

  const handleSubmit = (text: string) => {
    const feedbackObj = { id: Date.now(), text, timestamp: new
Date().toISOString() };
    socket.emit('submit_feedback', feedbackObj);
  };

  // ... render UI
};
```

Clean Out Over-Explaining Comments: Delete comments that just explain standard React logic (like `// Scroll to bottom` or `// Safely cleanup`). Keep comments strictly for business logic.

Remove Unused State & Imports: Clean up unused icons (`Sparkles`, `Phone`) and unused state variables (`explainedResources`).

Refactor Repetitive Logic: Replace long sequential `if` checks with standard JavaScript data configurations (`Array.map` / `Array.find`).

Custom Styling Touches: Swap generic Tailwind preset combinations (like `bg-stone-50 border-stone-200`) for custom CSS theme variables matching your site's unique design system.

Remove Dead Code & Unused State: Clean up unused state like `explainedResources`, unused imports, and fixed truncated JSX tags at the bottom.

Remove "AI Tells": Removed hyper-verbose comments (like `// Safely cleanup...` or `// Word-boundary match helper...`), artificial-sounding badges, and rigid internal function wrappers.
Decouple Search Engine Logic: Move keyword-matching functions out of the component scope so they don't re-instantiate on every React render cycle.
Add Quality-of-Life UX: Add an auto-focus `ref` so the input field automatically focuses when the chat opens, and an `aria-expanded` tag for accessibility.
Remove Form Field Reset Bugs: Currently, when a user switches between *Public* and *Anonymous* mode, `setName('')` is not called automatically, but `isAnonymous ? '' : name` is evaluated in the input. Syncing the name clear behavior on toggle ensures cleaner form state.
Clean Up Form Validation & Character Counter Accessibility: The `output` tag for the character counter uses `htmlFor="fb-message"`, which is great for semantic HTML! Adding dynamic styling (e.g., turning orange/red when near the 500-character limit) provides immediate visual feedback.
Smooth Scroll to Top of Feed on New Submission: When a user submits a note, automatically scrolling the message feed to the top (`feedRef.current?.scrollTo({ top: 0, behavior: 'smooth' })`) ensures they immediately see their newly posted note appearing at the top of the list.
Streamline Rating Selectors: Extracting the rating buttons into a small mapped configuration removes repetitive inline conditional logic for styling.

INDEX.HTML

If you want to ensure top-tier production readiness and Progressive Web App (PWA) standards, consider adding a favicon link and an explicit `theme-color` meta tag inside the `<head>`:

HTML

```
None
<!-- Favicon & PWA Theme -->
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="theme-color" content="#92400e" />
```

Here is the clean, fully valid JSON file configured specifically for **CivicBridge**:

JSON

```
None
{
  "name": "CivicBridge",
```

```
  "description": "A community resource directory connecting
residents of Cumming and Forsyth County, Georgia, with local
services, educational opportunities, and community programs.",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "COMMUNITY_RESOURCE_DIRECTORY",
    "RESOURCE_SEARCH_AND_FILTERING",
    "CATEGORY_BASED_NAVIGATION",
    "INTERACTIVE_COMMUNITY_RESOURCE_FINDER",
    "RESPONSIVE_WEB_DESIGN"
  ]
}
```

What to double-check in `metadata.json`:

1. **Valid JSON syntax:** Ensure there are no trailing commas (e.g., after the last item in `"majorCapabilities"`).
2. **File Location:** Place this file directly in your project root (`/metadata.json`) alongside `package.json` or `.gitattributes`.

Here are a few small additions to refine your `scripts` block for better cross-platform compatibility and production build checking:

1. **Cross-Platform `clean` Script:** Using `rm -rf` in your `clean` script works great on Linux/macOS, but will crash if someone on a Windows machine runs `npm run clean`. Using `rimraf` or Vite's built-in build directory wiping prevents cross-platform errors.
2. **Include Type-Checking in Build:** Running `tsc --noEmit` as part of your `build` step prevents broken types from slipping into production builds.

1. Path Alias Compatibility (`baseUrl`)

Depending on your TypeScript version, using path aliases like `@/*` without an explicit `"baseUrl"` can sometimes cause resolution warnings in certain IDE tools or third-party linters. Adding `"baseUrl": "."` ensures maximum tooling compatibility:

JSON

```
None
"baseUrl": ".",
```

```
"paths": {
  "@/*": ["./src/*"]
}
```

(Note: If your source files reside in `src/`, mapping `@/*` to `./src/*` is standard practice so `@/components` points directly into `src/components`).

2. Ensure `vite.config.ts` Matches Path Aliases

To use `@/` imports in your code, ensure your `vite.config.ts` also resolves the path alias using `path.resolve`.

Path Alias Mapping (`@`) In your `tsconfig.json`, we set up `@/*` to map to `./src/*`. In your `vite.config.ts`, you currently have:
TypeScript

```
None
'@': path.resolve(import.meta.dirname, '.')
```

This maps `@` to the project root directory rather than `src/`. Aligning it to `path.resolve(import.meta.dirname, './src')` ensures that imports like `@/components/FeedbackSection` resolve cleanly into your `src/` folder without needing extra relative subfolders.
Base Path for GitHub Pages (`base: '/Website-Design-Project/'`) Your base path is correctly set to your repository name (`/Website-Design-Project/`). This is critical for GitHub Pages deployments so assets (JS/CSS bundles) load from the correct subpath.
Node Compatibility (`import.meta.dirname`)`import.meta.dirname` is supported natively in modern Node.js versions (v20.11.0+ and v21.2.0+). If you ever build in an environment running an older Node runtime, using `fileURLToPath(new URL('.', import.meta.url))` is the standard ESM fallback.

1. Quick Fixes Needed in `App.tsx`

A. Remove Unused Icon Imports

You have several icons imported from `lucide-react` at the top of `App.tsx` that are declared but never rendered in the file. Cleaning them up reduces unused bundle bloat:

- `ChevronDown`
- `MapPin`
- `AlertCircle`
- `Copy`
- `ExternalLink`
- `BarChart3`
- `Github`
- `FileText`
- `ThumbsUp`
- `Bell`

B. Add `aria-expanded` & `aria-controls` to Hero Search Clear Button

On line 241, the search clear button should include explicit ARIA attributes so screen readers know it controls the search input state.

Recommended Additions (Copied Directly Below)

To ensure smooth cross-browser rendering, support Tailwind CSS v4 directives, and prevent accidental text overflow in resource cards, add these **3 quick utility enhancements** to the bottom of your CSS file:

CSS

```
None
/*
-------------------------------------------------------------------
-----------
   20. UTILITY OVERRIDES & TAILWIND V4 HELPERS

-------------------------------------------------------------------
----------- */

/* Prevent horizontal scroll bleed on mobile devices */
html, body {
  overflow-x: hidden;
}

/* Ensure smooth line truncation for card descriptions */
.line-clamp-2 {
```

```
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.line-clamp-3 {
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

/* Screen reader only utility class */
.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border-width: 0;
}
```

Optional React 19 `StrictMode` Wrap: For development safety (detecting side effects and deprecated code patterns), consider wrapping `<App/>` with `<React.StrictMode>`:

TypeScript

```
None
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { App } from './App.tsx';
import './index.css';
```

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

If you want `userConfirmations` to persist when a user refreshes the page, initialize the state from `localStorage`:

TypeScript

```
None
const [userConfirmations, setUserConfirmations] =
useState<Record<string, boolean>>(() => {
  try {
    const saved =
localStorage.getItem('civicbridge_user_confirmations');
    return saved ? JSON.parse(saved) : {};
  } catch {
    return {};
  }
});

const toggleUserConfirmation = (id: string) => {
  setUserConfirmations(prev => {
    const updated = { ...prev, [id]: !prev[id] };
    try {
      localStorage.setItem('civicbridge_user_confirmations',
JSON.stringify(updated));
    } catch {
      // LocalStorage access fallback
    }
    return updated;
  });
};
```

1. DRY Refactoring: Centralize Duplicate Verified Records

Many items in VERIFIED_RECORDS repeat identical placeholder structures.

- **Current Issue:** Entries like 'food-4' through 'food-15', 'repair-1' through 'repair-15', 'child-1' through 'child-15', and 'senior-1' through 'senior-15' copy-paste identical text blocks.
- **Improvement:** Generate baseline records using helper functions or mapping defaults to reduce boilerplate and file size.

TypeScript

```
None
// Example: Shared baseline for standard food/pantry verifications
const defaultFoodVerification: VerifiedRecordEntry = {
  isVerified: true,
  verificationDate: "Verified Nov 3, 2026",
  verificationSource: "Forsyth County Municipal & 501(c)(3)
Community Food Directory",
  website: "https://www.forsythco.com",
  eligibility: "Forsyth County families and individuals
experiencing temporary food insecurity.",
  sourceUrl: "https://www.forsythco.com",
  languages: ["English", "Spanish"],
  wheelchairAccessible: true,
  transitAccessible: true,
  intakeRequirements: [
    "Government photo ID",
    "Proof of Forsyth County residency (utility bill or lease
statement)"
  ],
};
```

2. Standardize CategoryType Syncing

- **Current Issue:** CATEGORIES uses id: 'schools', but CATEGORY_LABELS and some resource records call it 'schools' while others map differently (e.g., 'legal-crisis' exists in CATEGORY_LABELS but not in CATEGORIES).
- **Improvement:** Derive CategoryType directly from CATEGORIES using standard TypeScript type extraction to maintain strict type safety across the application:

TypeScript

```
None
export const CATEGORIES = [
  { id: 'all', label: 'All Resources' },
```

```
  { id: 'food', label: 'Food & Meals' },
  // ... rest of categories
] as const;

export type CategoryType = typeof CATEGORIES[number]['id'];

// Automatically build CATEGORY_LABELS from CATEGORIES
export const CATEGORY_LABELS: Record<string, string> =
Object.fromEntries(
  CATEGORIES.map(cat => [cat.id, cat.label])
);
```

3. Geographic District Resolution Logic

- **Current Issue:** In the factory function r(), district assignment relies on simple string matching against street names (address.includes('30041'), etc.). If an address formatting changes slightly (e.g., lowercase or abbreviated "Rd"), it defaults silently to 'Central Cumming'.
- **Improvement:** Case-insensitive address checking or explicitly passing district as a parameter when creating resources:

TypeScript

```
None
// Case-insensitive matching fallback helper
const addrUpper = address.toUpperCase();
if (['30041', 'BROWNS BRIDGE', 'JAMES BURGESS', 'TRAMMEL'].some(k
=> addrUpper.includes(k))) {
  district = 'East Forsyth';
}
```

4. Remove Redundant Flags in r() Helper

- **Current Issue:** Inside r(), isVerified is hardcoded to true on line 667, rendering the fallback logic redundant.
- **Improvement:** Clean up the internal variables in the helper:

TypeScript

```
None
// Streamlined lookup inside r()
const verifiedRecord = VERIFIED_RECORDS[id] || {};
```

```
return {
  id,
  name,
  category,
  categoryLabel: CATEGORY_LABELS[category] || 'Community Service',
  tagline,
  description,
  address,
  district,
  phone,
  website: verifiedRecord.website,
  hours,
  eligibility: verifiedRecord.eligibility || customEligibility ||
'Forsyth County residents and families.',
  cost,
  languages: verifiedRecord.languages || ['English', 'Spanish'],
  wheelchairAccessible: verifiedRecord.wheelchairAccessible ??
true,
  transitAccessible: verifiedRecord.transitAccessible ?? true,
  keyServices,
  intakeRequirements: verifiedRecord.intakeRequirements ||
['Contact provider for intake requirements'],
  verificationDate: verifiedRecord.verificationDate || 'Verified
Nov 2026',
  verificationSource: verifiedRecord.verificationSource || 'Forsyth
County Municipal Records Audit',
  sourceUrl: verifiedRecord.sourceUrl || customSourceUrl ||
'https://www.forsythco.com',
  isEmergency,
  isVerified: verifiedRecord.isVerified ?? true,
};
```

5. Performance Optimization: Convert Arrays to Lookup Maps

For features that search or filter resources frequently by id (e.g., rendering single resource details):

TypeScript

```
None
// Add a constant Map for O(1) lookups by ID
export const RESOURCE_MAP = new Map<string, ResourceItem>(
  COMMUNITY_RESOURCES.map(res => [res.id, res])
);
```

1. Simplify & DRY Up Data Management (mockData.ts)

- **Centralize Duplicate Verified Records:** Eliminate repeated boilerplate for items sharing identical validation properties. Use baseline objects or mapper functions:
- TypeScript

```
None
const DEFAULT_FOOD_VERIFICATION: VerifiedRecordEntry = {
  isVerified: true,
  verificationDate: "Verified Nov 3, 2026",
  verificationSource: "Forsyth County Municipal Audit",
  website: "https://www.forsythco.com",
  eligibility: "Forsyth County residents experiencing food
insecurity.",
  sourceUrl: "https://www.forsythco.com",
  languages: ["English", "Spanish"],
  wheelchairAccessible: true,
  transitAccessible: true,
  intakeRequirements: ["Government photo ID", "Proof of
residency"],
};
```

-
- **Single Source of Truth for Categories:** Extract CategoryType dynamically using as const so you never have mismatched categories (e.g., 'legal-crisis' present in labels but missing from main array):
- TypeScript

```
None
export type CategoryType = typeof CATEGORIES[number]['id'];
export const CATEGORY_LABELS: Record<string, string> =
Object.fromEntries(
  CATEGORIES.map(c => [c.id, c.label])
);
```

-
- **Add Fast Lookup Map for O(1) Performance:** Replace repeated .find() and .filter() array searches when matching single resource details:
- TypeScript

```
None
export const RESOURCE_MAP = new Map<string, ResourceItem>(
  COMMUNITY_RESOURCES.map(res => [res.id, res])
);
```

-

2. Improve Interactive Guide (CivicChatbot.tsx)

- **Prevent Stale States in Response Handler:** When matching follow-up queries like *"show me more"*, ensure the index pointer stays within range and avoids infinite clicks on empty results.
- **Improve Semantic Highlight Styling:** Combine the Learn More and Explain Further toggles into a unified expandable details container inside the chat cards to keep the message list cleaner and reduce vertical scrolling clutter.
- **Persist Chat Messages:** Store messages in sessionStorage or localStorage so users don't lose their search history when navigating across tabs or closing the chatbot modal.

3. Enhance Accessibility & Focus Traps (ResourceDetails.tsx)

- **Focus Restoral Fallback:** When restoring focus on component unmount, ensure there is a fallback to document.body if triggerElementRef.current is disconnected from the DOM.
- **Explicit Keyboard Trapping:** Use a dedicated library like focus-trap-react or clean up the Tab key listener to ensure invisible or disabled elements aren't accidentally prioritized in the focus loop.
- **Screen Reader Announcers:** Add an aria-live="polite" region when confirming information up to date (onQuickCheck) so screen readers announce the success status immediately.

4. Optimize Render Performance (ResourceCard.tsx)

- **Memoize Cards in Large Grids:** Wrap ResourceCard in React.memo to prevent re-rendering all 200+ cards when simple top-level filter states change.
- **Address Overflow & Text Truncation:** Ensure long service names or addresses gracefully wrap on small screen sizes (below 360px width) instead of breaking card layouts.

5. Standardize Form Validation & UX (FeedbackSection.tsx)

- **Controlled Input Reset on Tab Navigation:** Reset success/error banners when users change feedback filters or switch away from the feedback tab.
- **Accessible Radio Button Group:** Group the anonymity preferences inside a <fieldset> with a visible <legend> to comply fully with screen-reader guidelines.

6. Mobile Layout & Polish (Navbar.tsx & Footer.tsx)

- **Horizontal Scroll Indicators:** In mobile view, add subtle fade gradient edges to <nav> in Navbar.tsx to indicate to users that the navigation items scroll horizontally.
- **Dynamic Year in Footer:** Replace hardcoded content with dynamic runtime years where applicable to prevent copyright or system dates from growing stale over time.

1. Eliminate Boilerplate Duplication (`mockData.ts`)

Instead of repeating identical verification structures 15+ times for placeholder records, write a clean factory mapping or apply spread overrides:

mockData.ts
TypeScript

```
None
// Define a single baseline for recurring verification structures
const DEFAULT_VERIFICATION: VerifiedRecordEntry = {
  isVerified: true,
  verificationDate: "Verified Nov 3, 2026",
  verificationSource: "Forsyth County Municipal & 501(c)(3)
Community Audit",
  website: "https://www.forsythco.com",
  eligibility: "Forsyth County families and individuals
experiencing temporary hardship.",
  sourceUrl: "https://www.forsythco.com",
  languages: ["English", "Spanish"],
  wheelchairAccessible: true,
  transitAccessible: true,
  intakeRequirements: ["Government photo ID", "Proof of Forsyth
County residency"],
};

// Generate repetitive ranges dynamically or spread overrides
export const VERIFIED_RECORDS: Record<string, VerifiedRecordEntry>
= {
```

```
  'food-1': {
    ...DEFAULT_VERIFICATION,
    website: "https://www.theplaceofforsyth.org",
    sourceUrl: "https://www.theplaceofforsyth.org",
    eligibility: "Forsyth County residents facing food insecurity;
photo ID required.",
  },
  // Map repetitive batch IDs cleanly
  ...Object.fromEntries(
    Array.from({ length: 12 }, (_, i) => [`food-${i + 4}`,
DEFAULT_VERIFICATION])
  ),
};
```

2. Introduce Hand-Crafted Custom Hooks

AI generated code often puts all state and side-effects inside single large UI components. Moving key logic into custom React hooks makes the architecture modular and human-designed:

ResourceDetails.tsx+ 1

- **Modal Focus Trap Hook:** Extract the manual focus trapping from `ResourceDetails.tsx` into `useFocusTrap(ref, isOpen)`.
  ResourceDetails.tsx+ 1
- **Resource Matching Hook:** Extract query filtering and ranking logic from `CivicChatbot.tsx` into `useResourceSearch(query)`.
  CivicChatbot.tsx+ 1

TypeScript

```
None
// src/hooks/useFocusTrap.ts
import { useEffect, useRef } from 'react';

export function useFocusTrap<T extends HTMLElement>(isOpen:
boolean) {
  const containerRef = useRef<T>(null);
```

```
  const previousFocus = useRef<HTMLElement | null>(null);

  useEffect(() => {
    if (!isOpen) return;
    previousFocus.current = document.activeElement as HTMLElement;

    const focusables =
containerRef.current?.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea,
[tabindex]:not([tabindex="-1"])'
    );
    focusables?.[0]?.focus();

    return () => {
      previousFocus.current?.focus?.();
    };
  }, [isOpen]);

  return containerRef;
}
```

3. Consolidate Chat UI State (`CivicChatbot.tsx`)

AI outputs frequently manage UI expansion using separate `Set` states (`expandedResources`, `explainedResources`, `showAllResults`). Human engineers combine related UI toggle states into clean item-level components or unified state keys:

CivicChatbot.tsx
TypeScript

```
None
// Replace separate Sets with a single active item ID toggle
const [activeDetailsId, setActiveDetailsId] = useState<string |
null>(null);
const [activeExplanationId, setActiveExplanationId] =
useState<string | null>(null);
```

```
const toggleDetails = (id: string) => {
  setActiveDetailsId(prev => (prev === id ? null : id));
};
```

4. Add Real-World Defensive Code

Human code includes pragmatic fallbacks for unexpected runtime edge cases:

- **Phone Number Formatting Guard:** Handle invalid or missing phone strings cleanly across cards and modals:
  ResourceDetails.tsx+ 1
- TypeScript

```
None
const formatTelLink = (phone?: string) =>
  phone ? `tel:${phone.replace(/[^\d+]/g, '')}` : '#';
```

-
- **URL Truncation Guard:** Replace manual substring slicing (`.slice(0, 32)`) with URL parsing:
  ResourceDetails.tsx
- TypeScript

```
None
const formatDomain = (url: string) => {
  try {
    return new URL(url).hostname.replace('www.', '');
  } catch {
    return url;
  }
};
```

-

5. Performance Optimization (`ResourceCard.tsx`)

Human-written React applications memoize list cards to ensure smooth 60fps rendering during fast text searches:

TypeScript

```
None
import React, { memo } from 'react';

export const ResourceCard = memo(function ResourceCard({
  resource,
  onSelect,
  onCall,
}: ResourceCardProps) {
  // Render implementation...
});
```